Pashov Audit Group

Napier Security Review

February 6th 2026 - February 8th 2026

Contents

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Napier

Napier is a DeFi yield trading protocol built on Uniswap V4 hooks that enables splitting yield-bearing tokens into principal tokens (PT) and yield tokens (YT) for separate trading and management. The scope is 5 PRs that introduce vault deposit/withdraw event tracking, deposit cap verification module updates, Chainlink oracle label functions, and swap quote improvements with token conversion support.

5. Executive Summary

A time-boxed security review of the **napierfi/napier-v2**, **napierfi/napier-v2**, **napierfi/napier-v2**, **napierfi/napier-v2** and **napierfi/napier-v2** repositories was done by Pashov Audit Group, during which **Said**, **merlinboii**, **JCN** engaged to review **Napier**. A total of **5** issues were uncovered.

Protocol Summary	
Project Name	Napier
Protocol Type	Yield splitting
Timeline	February 6th 2026 - February 8th 2026

Review commit hashes:

- 9d5296fbebbd98e6595060c4c4c9a60ef215bc4863 (napierfi/napier-v2)
- 5df038e663686e7742b61df4ea88dbc2300ffd3d (napierfi/napier-v2)
- 6c917d0295602cb983da1d6bb11f06d8faf77d57 (napierfi/napier-v2)
- c7074838957dfddcd28fd618bfe1a3d1bc8fc74b (napierfi/napier-v2)
- 5959372afd67125f6758a2bb8fb779e6ad4f71f1 (napierfi/napier-v2)

Fixes review commit hash:

- a5d4c29f2f0fa07f8d6dec9ef82f2d60fb710216 (napierfi/napier-v2)

Scope

`Events.sol` `TokiHook.sol` `TokiHookLogic.sol` `VerifierModule.sol` `IChainlinkCompatibleAggregatorV3.sol` `TokiLinearChainlinkOracle.sol` `TokiTWAPChainlinkOracle.sol` `TokiQuoter.sol`

6. Findings

Findings count

Severity	Amount
Low	5
Total findings	**5**

Summary of findings

ID	Title	Severity	Status
[L-01]	Missing withdraw event in TokiHook._deposit	Low	Resolved
[L-02]	Unused stack variable and redundant SLOAD in `DepositCapVerifierModule::initialize`	Low	Resolved
[L-03]	Inconsistent oracle label values	Low	Resolved
[L-04]	Backward-compatible `quoteSwapPt` alters `amountIn` return for PT purchase	Low	Resolved
[L-05]	Inconsistent `amountIn` between PT and YT quote functions	Low	Resolved

Low findings

[L-01] Missing withdraw event in TokiHook._deposit

Description

`TokiHook._deposit()` emits `VaultDeposit` after vault funding, but can later redeem `refundShares` from the vault without emitting `VaultWithdraw`.

In the refund path, reserves are reduced and `LibRehypothecation.redeemFromVault()` is executed, yet no withdraw event is produced.

```
function _deposit(DepositParams memory params) internal returns (DepositReturnData memory
returnData) {
      //--- SNIPPED ---

      uint256 rawAmount0;
      {
          uint256 refundShares0;
          (rawAmount0, refundShares0) = _calculateRefundAndRawAmount(
              CalculateRefundAndRawAmountParams({...})
          );

          // Update memory state
          params.state.reserves = params.state.reserves.sub(refundShares0.toUint128(), 0);

          // Assumption: vault must redeem exactly the requested amount of shares
          LibRehypothecation.redeemFromVault(
              params.vault0, Currency.unwrap(params.key.currency0), refundShares0,
params.refundReceiver
          );

          //@audit missing `emitVaultWithdraw(...)` for both currency0 and currency1 branches
      }

      //--- SNIPPED ---
  }
```

Consider emitting `VaultWithdraw` after each refund redemption.

[L-02] Unused stack variable and redundant SLOAD in `DepositCapVerifierModule::initialize`

Description

In `DepositCapVerifierModule::initialize`, the decoded `cap` value is stored in a local variable, written to storage, and then the event is emitted using the storage variable instead of the already available stack variable. This results in an unnecessary `SLOAD` and a redundant local variable:

```
    function initialize() external override initializer {
        (, bytes memory args) = abi.decode(LibClone.argsOnClone(address(this)), (address,
bytes));
        uint256 cap = abi.decode(args, (uint256));
        s_depositCap = cap;

        emit DepositCapUpdated(0, s_depositCap);
    }
```

Recommendation

Consider emitting the `cap` stack variable directly instead of reading from storage when emitting the event.

[L-03] Inconsistent oracle label values

Description

A new `label()` view function was added to the following oracle contracts:

```
contract TokiLinearChainlinkOracle is IChainlinkCompatibleAggregatorV3, Initializable {
...
    function label() external view returns (bytes32) {
        return "linear";
    }
}

contract TokiTWAPChainlinkOracle is IChainlinkCompatibleAggregatorV3, Initializable {
...
    function label() external view returns (bytes32) {
        return "twap";
    }
}
```

These labels do not match the contract names and are therefore less descriptive than the labels used elsewhere in the codebase:

```
contract ERC4626InfoResolver is VaultInfoResolver {
...
    function label() public pure override returns (bytes32) {
        return "ERC4626InfoResolver";
    }
```

Returning generic values such as "linear" or "twap" reduces clarity and makes off-chain validation and UI-level identification less reliable, as the data source used for the oracle is ambiguous (a "twap" oracle can have Uniswap, Chainlink, or some other protocol as a data source).

Consider updating the new `labels` to return more descriptive identifiers (e.g., matching the contract's name).

[L-04] Backward-compatible `quoteSwapPt` alters `amountIn` return for PT purchase

Description

PR #562 adds a `quoteSwapPt(QuoteV4SwapParams)` overload explicitly marked as left for compatibility with frontend. However, in the same PR, `_quoteV4SwapPt` was changed to return `params.amount` instead of the actual consumed amount, meaning the backward-compatible overload returns different values than the original function it replaces.

```
    function _quoteV4SwapPt(QuoteV4SwapParams memory params) internal view returns
(QuoteSwapResult memory result) {
        // ...

        if (params.zeroForOne) {
            // underlying -> PT swap
            // Note: due to binary search, the `underlyingAmount` is usually less than the
`params.amount`
            result.amountIn = params.amount; // User pays underlying (positive)
            result.amountOut = uint256(principals); // User receives PT (positive)
        } else {
            // PT -> underlying swap
            result.amountOut = uint256(underlyingAmount); // User receives underlying (positive)
            result.amountIn = params.amount; // User pays PT (positive)
        }
        // ...
```

Due to binary search, `uint256(-underlyingAmount) <= params.amount`. The old code returned the tighter estimate; the new code returns the ceiling.

```
    /// @dev left for compatibility with frontend
    function quoteSwapPt(QuoteV4SwapParams calldata params)
        external
        view
        checkPoolKey(params.poolKey)
        returns (QuoteSwapResult memory)
    {
        return _quoteV4SwapPt(params);
    }
```

A frontend that previously consumed amountIn from quoteSwapPt as the amount the user will actually spend now receives an inflated value.

Consider restoring the actual consumed amount in `_quoteV4SwapPt` for the buy direction to preserve backward-compatible return:

```
  if (params.zeroForOne) {

-       result.amountIn = params.amount; // User pays underlying (positive)
+      result.amountIn = uint256(-underlyingAmount); // actual consumed
      result.amountOut = uint256(principals);
  }
```

[L-05] Inconsistent `amountIn` between PT and YT quote functions

Description

Across PRs #561 and #562, the amountIn field in `QuoteSwapResult` has different meanings depending on which function is called, which direction, and which token type is used. This creates integration confusion for third parties / frontends who expect uniform definitions.

The table outlines the varying meanings of the `amountIn` field in the `QuoteSwapResult` across different functions and scenarios. It highlights the inconsistencies that arise when using the `quoteSwapPt` and `quoteSwapYt` functions, which can lead to confusion for third-party integrations and frontends expecting uniform definitions.

For the `quoteSwapPt(QuoteSwapParams)` function, when the direction is set to buy (with `zeroForOne=true`), the `amountIn` represents `params.amount`, which is the maximum input allowed. Conversely, when the direction is set to sell (with `zeroForOne=false`), `amountIn` signifies `params.amount` as the exact PT input required.

In the case of the `quoteSwapPt(QuoteV4SwapParams)` function, the `amountIn` also indicates `params.amount` as the maximum input when the direction is buy.

For the `quoteSwapYt(QuoteSwapParams)` function, when buying with the underlying token, the `amountIn` reflects the actual underlying consumed, which is less than the maximum allowed. If the token is non-underlying, `amountIn` corresponds to `params.amount`, representing the maximum input.

When using the `quoteSwapYt(QuoteV4SwapParams)` function in a buy scenario, the `amountIn` again indicates the actual underlying consumed, which is less than the maximum. In a sell scenario with `quoteSwapYt(QuoteSwapParams)`, the `amountIn` is defined as the exact YT input. The same holds true for the `quoteSwapYt(QuoteV4SwapParams)` function when selling.

These discrepancies in the definition of `amountIn` across different functions and scenarios underscore the need for clarity and consistency to avoid integration issues.

The root cause for the YT inconsistency is the explicit workaround in `quoteSwapYt(QuoteSwapParams)`.

```
function quoteSwapYt(QuoteSwapParams calldata params)
    external
    view
    checkPoolKey(params.poolKey)
    returns (QuoteSwapResult memory result)
{
    PrincipalToken pt = PrincipalToken(Currency.unwrap(params.poolKey.currency1));
    PrincipalTokenQuoter quoter = principalTokenQuoter();

    if (params.zeroForOne) {
        // Buy YT with token
        // Convert token to underlying token first
```

```
            uint256 amount0 = quoter.vaultPreviewDeposit(pt, params.token, params.amount);
            result = _quoteV4SwapYt(
                QuoteV4SwapParams({
                    poolKey: params.poolKey,
                    zeroForOne: params.zeroForOne,
                    amount: amount0.toUint128(),
                    approx: params.approx
                })
            );
            // Workaround to keep the same behavior as the old quoteSwapYt
            // The actual spent amount is usually less than the amount of underlying token spent
because of the binary search
            if (!params.token.eq(pt.underlying())) {
>>>             result.amountIn = params.amount;
            }
        }
       // ...
}
```

This means:

- User calling `quoteSwapYt` with `token = underlying` gets the actual spent amount.
- User calling `quoteSwapYt` with `token = baseToken` gets the maximum amount.
- User calling `quoteSwapPt` with `token = underlying` gets the maximum amount (inconsistent with the YT underlying path).

Consider standardizing the definitions of `amountIn` across all quote functions.